Decisiones técnicas

Mini sistema de inventario

- Auth JWT Bearer + guard global con opt-out: todo privado por defecto (‘APP_GUARD JwtAuthGuard’ en ‘app.module.ts’), solo ‘POST /auth/login’ marcado ‘@Public()’. Estrategia ‘jwt.strategy.ts’ valida ‘HS256’ y expone ‘{id,email}’; el frontend inyecta ‘Bearer’ y en ‘401’ limpia token y redirige a login.

- Validación y seguridad HTTP: ‘ValidationPipe(whitelist, forbidNonWhitelisted, transform)’, ‘helmet()’, ‘setGlobalPrefix('api')’, CORS restringido a ‘CORS_ORIGIN’ (‘main.ts’).

- Paginación uniforme: DTO ‘page=1/limit=10/max 100’ + envoltorio ‘{data,meta}’ vía ‘buildPaginated()’ en productos y kardex.

- Stock = suma de movimientos, protegido con bloqueo pesimista: ‘movements.service.ts’ corre en transacción con ‘pessimistic_write’ (‘SELECT ... FOR UPDATE’); valida ‘stock<0 → 422’ y tope int PG ‘>2147483647 → 422’; devuelve ‘stockAfter’.

- Migraciones versionadas, ‘synchronize:false’: el esquema solo cambia vía ‘migrations/’; seed de admin idempotente (‘bcrypt cost 12’, se omite si el email existe).

- **Frontend sin ‘VITE_*’:** cliente con ‘baseURL:'/api'‘ relativo; en dev proxy Vite → ‘:3000’, en prod nginx ‘location /api/ → backend:3000’ + fallback SPA y cache ‘immutable’ en assets.

RPA de consulta de antecedentes

- Modularidad: lector, validador, cliente HTTP y generador de reporte son módulos independientes; ‘main.py’ solo los orquesta. El cliente recibe la sesión HTTP y la función de espera por inyección, lo que permite probar reintentos y servicio caído sin red y sin esperar.

- Resiliencia: cada cédula se procesa aislada; ante cualquier fallo (incluida una excepción inesperada) se registra, se marca ‘error’ y se continúa con la siguiente.

- Docker: una sola imagen sirve al RPA y al servicio simulado. Los servicios del RPA están en un perfil de Compose, no publican puertos y se comunican por la red interna, de modo que no entran en conflicto con el Mini Sistema de Inventario.